

```
// =====
// App.jsx - Frontend React connecté au backend Node.js
// Remplacer l'ancien App.jsx par ce fichier
// =====

import { useState, useEffect } from "react";

const API = "http://localhost:3001/api";

const FIELD_LABELS = {
  banque: "Banque",
  tpe: "TPE",
  espece: "Espèce",
  paiementLivraison: "Paiement à la livraison",
  steLivraison: "STE de livraison",
};

const INITIAL_FIELDS = { banque: "", tpe: "", espece: "", paiementLivraison: "",
  steLivraison: "" };

const JUSTIFICATION_TYPES = [
  { value: "manque", label: "Manque", icon: "📉" },
  { value: "avoir_facture", label: "Avoir facture", icon: "🧾" },
  { value: "depense", label: "Dépense", icon: "💸" },
  { value: "reste_payer", label: "Reste à payer", icon: "🕒" },
  { value: "reste_deposer", label: "Reste à déposer (STE Livraison)", icon: "📦" }
];

const fmt = (v) => new Intl.NumberFormat("fr-TN", { style: "currency", currency:
const fnum = (v) => parseFloat(v) || 0;

export default function App() {
  const [step, setStep] = useState(1);
  const [date, setDate] = useState(new Date().toISOString().slice(0, 10));
  const [adminFields, setAdminFields] = useState({ ...INITIAL_FIELDS });
  const [finFields, setFinFields] = useState({ ...INITIAL_FIELDS });
  const [etatCourant, setEtatCourant] = useState(null);
  const [newJust, setNewJust] = useState({ type: "", montant: "", note: "" });
  const [toast, setToast] = useState(null);
  const [loading, setLoading] = useState(false);
  const [testEmailStatus, setTestEmailStatus] = useState(null);

  const showToast = (msg, color = "#22c55e") => {
    setToast({ msg, color });
    setTimeout(() => setToast(null), 4000);
  };
};
```

```

// — Étape 1 : Ouvrir l'état —
const handleAdminOpen = async () => {
  for (const k of Object.keys(adminFields)) {
    if (adminFields[k] === "") return showToast("Veuillez remplir tous les cham
  }
  setLoading(true);
  try {
    const res = await fetch(`${API}/etats`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ date, adminFields }),
    });
    const data = await res.json();
    setEtatCourant(data);
    setStep(2);
    showToast("État ouvert — en attente du financier.");
  } catch {
    showToast("Erreur connexion serveur. Vérifiez que node server.js tourne.",
  }
  setLoading(false);
};

// — Étape 2 : Valider —
const handleFinancierValidate = async () => {
  for (const k of Object.keys(finFields)) {
    if (finFields[k] === "") return showToast("Veuillez remplir tous les champs
  }
  setLoading(true);
  try {
    const res = await fetch(`${API}/etats/${etatCourant.id}/valider`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ finFields }),
    });
    const data = await res.json();
    setEtatCourant(data);
    if (data.status === "closed") {
      setStep(4);
      showToast("✅ Tous les montants correspondent. État fermé !", "#22c55e");
    } else {
      setStep(3);
      showToast("⚠️ Des écarts ont été détectés. Justification requise.", "#f59
    }
  } catch {
    showToast("Erreur connexion serveur.", "#ef4444");
  }
}

```

```

    setLoading(false);
};

// — Étape 3 : Ajouter justification —
const addJustification = async () => {
  if (!newJust.type || !newJust.montant || !newJust.note)
    return showToast("Remplissez tous les champs de justification.", "#ef4444")
  setLoading(true);
  try {
    const res = await fetch(`${API}/etats/${etatCourant.id}/justifications`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(newJust),
    });
    const data = await res.json();
    setEtatCourant(data);
    setNewJust({ type: "", montant: "", note: "" });
    if (data.status === "closed") {
      setStep(4);
      showToast("✅ Tous les écarts sont justifiés. État fermé !", "#22c55e");
    } else {
      showToast("Justification ajoutée.", "#a78bfa");
    }
  } catch {
    showToast("Erreur connexion serveur.", "#ef4444");
  }
  setLoading(false);
};

const removeJustification = async (jid) => {
  try {
    const res = await fetch(`${API}/etats/${etatCourant.id}/justifications/${jid}`);
    const data = await res.json();
    setEtatCourant(data);
  } catch {
    showToast("Erreur suppression.", "#ef4444");
  }
};

const testEmail = async () => {
  setTestEmailStatus("sending");
  try {
    const res = await fetch(`${API}/test-email`, { method: "POST" });
    const data = await res.json();
    setTestEmailStatus(data.success ? "ok" : "error");
    showToast(data.success ? "✅ Email de test envoyé !" : `❌ ${data.error}`,
  } catch {

```

```

        setTestEmailStatus("error");
        showToast("Erreur connexion serveur.", "#ef4444");
    }
    setTimeout(() => setTestEmailStatus(null), 3000);
};

const reset = () => {
    setStep(1);
    setDate(new Date().toISOString().slice(0, 10));
    setAdminFields({ ...INITIAL_FIELDS });
    setFinFields({ ...INITIAL_FIELDS });
    setEtatCourant(null);
    setNewJust({ type: "", montant: "", note: "" });
};

const ecarts = etatCourant?.ecarts || {};
const justifications = etatCourant?.justifications || [];
const totalEcart = Object.values(ecarts).reduce((s, v) => s + Math.abs(v), 0);
const totalJustifie = justifications.reduce((s, j) => s + j.montant, 0);
const restant = Math.max(0, totalEcart - totalJustifie);

return (
    <div style={{ fontFamily: "'Crimson Pro','Georgia',serif", background: "#0f11
        <style>{`
            @import url('https://fonts.googleapis.com/css2?family=Crimson+Pro:wght@30
            *{box-sizing:border-box;margin:0;padding:0;}
            ::-webkit-scrollbar{width:6px;}::-webkit-scrollbar-track{background:#0f11
            .inp{background:#1a1a2e;border:1px solid #2e2a3e;border-radius:6px;paddin
            .inp:focus{border-color:#8b5cf6;}
            .ig label{font-size:11px;letter-spacing:0.12em;text-transform:uppercase;c
            .ig{margin-bottom:14px;}
            .btn{border:none;border-radius:7px;padding:12px 28px;font-size:15px;font-
            .bp{background:linear-gradient(135deg,#7c3aed,#a855f7);color:#fff;}
            .bp:hover{transform:translateY(-1px);box-shadow:0 6px 20px #7c3aed44;}
            .bs{background:linear-gradient(135deg,#15803d,#22c55e);color:#fff;}
            .bs:hover{transform:translateY(-1px);box-shadow:0 6px 20px #22c55e44;}
            .bg{background:#252438;color:#8a7f9a;}
            .card{background:#161622;border:1px solid #252438;border-radius:12px;padd
            .two{display:grid;grid-template-columns:1fr 1fr;gap:16px;}
            .step-badge{display:inline-flex;align-items:center;gap:8px;background:#1e
            .st{font-size:22px;font-weight:600;color:#e8e0d0;margin-bottom:6px;}
            .ss{font-size:14px;color:#6b6380;margin-bottom:24px;font-family:'JetBrain
            hr{border:none;border-top:1px solid #252438;margin:20px 0;}
            .er{display:flex;justify-content:space-between;align-items:center;padding
            .ep{background:#0d231855;border:1px solid #16532d44;color:#4ade80;}
            .en{background:#2d0d0d55;border:1px solid #7f1d1d44;color:#f87171;}
            .pb{background:#1a1a2e;border-radius:20px;height:8px;overflow:hidden;marg

```

```

.pbi{height:100%;border-radius:20px;transition:width 0.4s;}
.ji{display:flex;align-items:flex-start;justify-content:space-between;gap
 stepper{display:flex;align-items:center;gap:0;margin-bottom:36px;}
.sd{width:32px;height:32px;border-radius:50%;display:flex;align-items:cen
.sa{background:#7c3aed;color:#fff;box-shadow:0 0 12px #7c3aed88;}
.sdo{background:#15803d;color:#fff;}
.sp{background:#252438;color:#6b6380;}
.sl{flex:1;height:2px;background:#252438;}
.sld{background:#15803d;}
.toast{position:fixed;bottom:30px;left:50%;transform:translateX(-50%);pad
.loader{display:inline-block;width:14px;height:14px;border:2px solid rgba
@keyframes spin{to{transform:rotate(360deg)}}
.notif-badge{background:#1e2d1e;border:1px solid #16532d;border-radius:6p
@media (max-width:600px){.two{grid-template-columns:1fr;}.er{flex-directio
`}</style>

```

```

{toast && <div className="toast" style={{ background: toast.color }}>{toast

```

```

<div style={{ maxWidth: 820, margin: "0 auto", padding: "40px 20px" }}>

```

```

{/* Header */}

```

```

<div style={{ marginBottom: 36, display: "flex", justifyContent: "space-b
  <div>
    <div style={{ fontSize: 11, fontFamily: "'JetBrains Mono',monospace",
    <h1 style={{ fontSize: 32, fontWeight: 700, background: "linear-gradi
      État de Vente
    </h1>
    <div style={{ fontSize: 14, color: "#5a5070", fontFamily: "'JetBrains
  </div>
  <button className="btn bg" style={{ fontSize: 12, padding: "8px 16px",
    {testEmailStatus === "sending" ? <>Test email<span className="loader"
  </button>
</div>

```

```

{/* Notification info */}

```

```

{step >= 2 && etatCourant?.status === "open" && (
  <div className="notif-badge" style={{ marginBottom: 20 }}>
    <span>🔔</span>
    <span>Notification automatique si cet état reste ouvert 24h – Admin &
  </div>
)}

```

```

{/* Stepper */}

```

```

<div className="stepper">
  {[1, 2, 3].map((s, i) => (
    <div key={s} style={{ display: "contents" }}>
      <div className={`sd ${step > s ? "sdo" : step === s ? "sa" : "sp"}`

```

```

        {i < 2 && <div className={`sl ${step} > s ? "sld" : ""}`} />
      </div>
    )}}
  </div>

  {/* — STEP 1 — */}
  {step === 1 && (
    <div className="card">
      <div className="step-badge">O Étape 1 / Administrateur</div>
      <div className="st">Ouverture de l'état</div>
      <div className="ss">Saisir les montants prévisionnels</div>
      <div className="ig">
        <label>Date</label>
        <input type="date" className="inp" value={date} onChange={e => setD
      </div>
      <div className="two">
        {Object.keys(INITIAL_FIELDS).map(k => (
          <div className="ig" key={k}>
            <label>{FIELD_LABELS[k]}</label>
            <input type="number" className="inp" placeholder="0.000" step="
              value={adminFields[k]} onChange={e => setAdminFields({ ...adm
          </div>
        )}}
      </div>
      <hr />
      <div style={{ display: "flex", justifyContent: "flex-end" }}>
        <button className="btn bp" onClick={handleAdminOpen} disabled={load
          {loading ? <>Ouverture...<span className="loader" /></> : "Ouvrir
        </button>
      </div>
    </div>
  )}

  {/* — STEP 2 — */}
  {step === 2 && (
    <div className="card">
      <div className="step-badge">O Étape 2 / Financier</div>
      <div className="st">Validation financière</div>
      <div className="ss">Montants réels constatés – {date}</div>
      <div className="two">
        <div>
          <div style={{ fontSize: 11, fontFamily: "'JetBrains Mono',monospa
            {Object.keys(INITIAL_FIELDS).map(k => (
              <div key={k} style={{ display: "flex", justifyContent: "space-b
                <span style={{ color: "#8a7f9a" }}>{FIELD_LABELS[k]}</span>
                <span style={{ fontFamily: "'JetBrains Mono',monospace", colo
              </div>
            )}}
          </div>
        </div>
      </div>
    </div>
  )}

```

```

    )))
  </div>
  <div>
    <div style={{ fontSize: 11, fontFamily: "'JetBrains Mono', monospa
      {Object.keys(INITIAL_FIELDS).map(k => (
        <div className="ig" key={k}>
          <label>{FIELD_LABELS[k]}</label>
          <input type="number" className="inp" placeholder="0.000" step
            value={finFields[k]} onChange={e => setFinFields({ ...finFi
          </div>
        )}}
      </div>
    </div>
  </div>
  <hr />
  <div style={{ display: "flex", justifyContent: "space-between" }}>
    <button className="btn bg" onClick={() => setStep(1)}>← Retour</but
    <button className="btn bs" onClick={handleFinancierValidate} disabl
      {loading ? <>Validation...<span className="loader" /></> : "Valid
    </button>
  </div>
</div>
))

```

{/\* — STEP 3 — \*/}
 {step === 3 && (
 <>
 <div className="card">
 <div className="step-badge" style={{ borderColor: "#92400e", color:
 <div className="st">Analyse des écarts</div>
 <div className="ss">Justifier chaque écart pour fermer l'état</div>

 {Object.keys(ecarts).filter(k => Math.abs(ecarts[k]) > 0.001).map(k
 <div key={k} className={`er \${ecarts[k] > 0 ? "ep" : "en"}`}>
 <span>{FIELD\_LABELS[k]}</span>
 <div style={{ display: "flex", gap: 16, flexWrap: "wrap" }}>
 <span>Admin: {fmt(etatCourant.adminFields[k])}</span>
 <span>Fin: {fmt(etatCourant.finFields[k])}</span>
 <span style={{ fontWeight: 600 }}>Écart: {ecarts[k] > 0 ? "+"
 </div>
 </div>
 </div>
 </div>
 </div>
 )}

```

  <div style={{ background: "#1a1a2e", border: "1px solid #2e2a3e", b
    <div style={{ display: "flex", justifyContent: "space-between", f
      <span style={{ color: "#8a7f9a", fontFamily: "'JetBrains Mono',
      <span style={{ color: "#fbbf24", fontFamily: "'JetBrains Mono',
    </div>
  
```

```

    <div style={{ display: "flex", justifyContent: "space-between", f
      <span style={{ color: "#8a7f9a", fontFamily: "'JetBrains Mono',
      <span style={{ color: "#4ade80", fontFamily: "'JetBrains Mono',
    </div>
    <div className="pb">
      <div className="pbi" style={{
        width: `${Math.min(100, totalEcart > 0 ? (totalJustifie / tot
        background: restant < 0.001 ? "#22c55e" : "linear-gradient(90
      }} />
    </div>
    <div style={{ fontSize: 12, fontFamily: "'JetBrains Mono',monospa
      Restant: {fmt(restant)}
    </div>
  </div>
</div>

<div className="card">
  <div style={{ fontSize: 11, fontFamily: "'JetBrains Mono',monospace
  <div className="two">
    <div className="ig">
      <label>Type</label>
      <select className="inp" value={newJust.type} onChange={e => set
        <option value="">- Sélectionner -</option>
        {JUSTIFICATION_TYPES.map(t => <option key={t.value} value={t.
      </select>
    </div>
    <div className="ig">
      <label>Montant (TND)</label>
      <input type="number" className="inp" placeholder="0.000" step="
        value={newJust.montant} onChange={e => setNewJust({ ...newJus
      </div>
    </div>
    <div className="ig">
      <label>Note / Commentaire</label>
      <textarea className="inp" rows={2} placeholder="Détail..." value=
        onChange={e => setNewJust({ ...newJust, note: e.target.value })
      </div>
    <button className="btn bp" onClick={addJustification} disabled={loa
      {loading ? <>Ajout...<span className="loader" /></> : "+ Ajouter
    </button>
  </div>

{justifications.length > 0 && (
  <div className="card">
    <div style={{ fontSize: 11, fontFamily: "'JetBrains Mono',monospa
      JUSTIFICATIONS ({justifications.length})
    </div>

```



```

    {justifications.map(j => {
      const t = JUSTIFICATION_TYPES.find(x => x.value === j.type);
      return (
        <div className="ji" key={j.id}>
          <div style={{ flex: 1 }}>
            <div style={{ display: "flex", alignItems: "center", gap:
              <span>{t?.icon}</span>
              <span style={{ fontWeight: 600, color: "#a78bfa" }}>{t?
                <span style={{ fontFamily: "'JetBrains Mono',monospace"
              </div>
              <div style={{ fontSize: 13, color: "#8a7f9a" }}>{j.note}<
            </div>
            <button onClick={() => removeJustification(j.id)} style={{
          </div>
        });
      })}
    </div>
  )}
</>
)}

{/* — STEP 4 — */}
{step === 4 && (
  <div className="card" style={{ textAlign: "center", padding: "60px 32px"
    <div style={{ fontSize: 64, marginBottom: 20, animation: "pop 0.5s cu
    <style>{@keyframes pop{from{transform:scale(0);opacity:0}to{transfor
    <h2 style={{ fontSize: 28, fontWeight: 700, marginBottom: 10 }}>État
    <p style={{ color: "#6b6380", fontFamily: "'JetBrains Mono',monospace
      {justifications.length === 0
        ? "Tous les montants correspondent parfaitement."
        : `${justifications.length} justification(s) validée(s).`
    </p>
    <div style={{ background: "#1a1a2e", border: "1px solid #2e2a3e", bor
    <div style={{ fontSize: 11, fontFamily: "'JetBrains Mono',monospace
      {Object.keys(FIELD_LABELS).map(k => (
        <div key={k} style={{ display: "flex", justifyContent: "space-bet
          <span style={{ color: "#8a7f9a" }}>{FIELD_LABELS[k]}</span>
          <span style={{ fontFamily: "'JetBrains Mono',monospace", color:
            {fmt(etatCourant?.finFields?.[k] ?? adminFields[k])}
          </span>
        </div>
      </div>
    )}}
    <div style={{ display: "flex", justifyContent: "space-between", pad
      <span>Total</span>
      <span style={{ fontFamily: "'JetBrains Mono',monospace", color: "
        {fmt(Object.keys(FIELD_LABELS).reduce((s, k) => s + fnum(etatCo
      </span>

```

```
        </div>
    </div>
    <button className="btn bp" onClick={reset}>+ Nouvel état de vente</bu
</div>
    ) }
</div>
</div>
);
}
```